FADENA
FACULTAD DE DEFENSA NACIONAL

Universidad de la Defensa Nacional

## Facultad de Defensa Nacional

## Licenciatura en Ciberdefensa

# Sistemas de Tratamiento de Datos

## Introducción a las Expresiones Regulares (REGEX)

**Año:** 2025

## ¿Qué son las Expresiones Regulares?

Una **Expresión Regular**, comúnmente abreviada como **REGEX** o **RegExp**, es una secuencia de caracteres que define un patrón de búsqueda. Haciendo una analogía simple, pensarla como una versión extremadamente potente de la función "Buscar" (Ctrl+F) que usamos para buscar en editores de texto.

En lugar de buscar un texto literal y estático (como la palabra "hola"), una expresión regular te permite buscar patrones complejos. Por ejemplo, podríamos buscar:

- Cualquier número de teléfono, sin importar si tiene guiones, espacios o paréntesis.
- Todas las direcciones de correo electrónico válidas dentro de un documento.
- Líneas en un archivo de código que comiencen con la palabra import y terminen con un punto y coma.
- Palabras que empiecen con "a" y terminen con "o", sin importar lo que haya en medio.

### ¿Por qué son tan útiles?

Las expresiones regulares se utilizan principalmente para tres tareas:

1. **Validación:** Comprobar si un texto cumple con un formato específico (ej: un formulario de registro que valida si la contraseña es segura).
2. **Búsqueda y extracción:** Encontrar y extraer fragmentos de texto que coinciden con un patrón (ej: sacar todos los enlaces de una página web).
3. **Sustitución:** Buscar un patrón y reemplazarlo por otro texto (ej: anonimizar todos los números de DNI en un documento).

## Una Breve Historia

El concepto de las expresiones regulares nació en la década de 1950 en el campo de la informática teórica y las matemáticas. Sin embargo, su popularización llegó con las herramientas del sistema operativo Unix, especialmente con el editor de texto ed y la utilidad grep (cuyo nombre significa globally search for a regular expression and print), desarrolladas por Ken Thompson en los laboratorios Bell. Desde entonces, se han convertido en una característica estándar en la mayoría de los lenguajes de programación y herramientas de texto.

### Conceptos Fundamentales

Para entender las REGEX, primero debemos diferenciar entre dos tipos de caracteres dentro de un patrón: los **literales** y los **metacaracteres**.

### Caracteres Literales

Un carácter literal es simplemente un carácter que se representa a sí mismo. La expresión regular más simple es una secuencia de caracteres literales.

**Ejemplo:**
El patrón gato buscará exactamente la secuencia de caracteres "g", "a", "t", "o".

| **Texto de prueba** | **¿Coincide con gato?** |
|---|---|
| El **gato** negro. | Sí |
| Me gustan los **gato**s. | Sí |
| El perro y el gato. | Sí |
| Ágata | No |
| El ratón | No |

Como puedes ver, es una búsqueda literal, tal como funciona Ctrl+F.

Metacaracteres: Los Caracteres con Superpoderes

Aquí es donde comienza la magia. Los **metacaracteres** son caracteres especiales que no se representan a sí mismos, sino que tienen un significado especial dentro del motor de REGEX. Le dan instrucciones sobre cómo debe realizarse la búsqueda.

Los metacaracteres más comunes son:

| **Metacarácter** | **Nombre** | **Función (resumen)** |
|---|---|---|
| . | Punto (Comodín) | Coincide con cualquier caracter. |
| \ | Barra invertida (Escape) | Anula el significado especial de un metacaracter. |
| \| | Barra vertical (Alternancia) | Actúa como un operador "OR". |
| * | Asterisco (Cuantificador) | Coincide cero o más veces. |
| + | Más (Cuantificador) | Coincide una o más veces. |
| ? | Interrogación (Cuantificador) | Coincide cero o una vez (opcional). |
| {} | Llaves (Cuantificador) | Coincide un número específico de veces. |
| [] | Corchetes (Clase de caracteres) | Coincide con cualquier carácter dentro de ellos. |

| | | |
|---|---|---|
| () | Paréntesis (Grupo) | Agrupa expresiones y captura coincidencias. |
| ^ | Acento circunflejo (Ancla) | Coincide con el inicio de una línea. |
| $ | Dólar (Ancla) | Coincide con el fin de una línea. |

A continuación, exploraremos cada uno de estos metacaracteres en detalle.

### Anclas y Delimitadores de Palabra

Las anclas no coinciden con ningún carácter, sino con una **posición** específica dentro del texto: el principio, el final o el límite de una palabra.

### Anclas de Inicio y Fin: ^ y $

Estos metacaracteres son extremadamente útiles para asegurar que tu patrón coincide en una ubicación específica de una línea.

- ^ (Acento circunflejo): Coincide con el **inicio** de una línea o del texto.
- $ (Monetario): Coincide con el **final** de una línea o del texto.

**Ejemplos:**

1. **Patrón: ^Hola**
   - Coincide con "**Hola**, mundo".
   - No coincide con "Adiós y **Hola**". La palabra "Hola" no está al principio.
2. **Patrón: fin$**
   - Coincide con "Este es el **fin**".
   - No coincide con "El **fin** del mundo". La palabra "fin" no está al final.

3. **Patrón: ^Solo esto$**
    - Este patrón es muy estricto. Coincidirá únicamente si la línea contiene *exactamente* el texto "Solo esto" y nada más.

Delimitadores de Palabra: \b

El metacaracter \b es otra ancla que coincide con una posición, no con un carácter. Específicamente, coincide con un **límite de palabra** (word boundary). Un límite de palabra es la posición entre un carácter de palabra (\w, que veremos más adelante) y un carácter que no es de palabra (\W), como un espacio, un punto o el inicio/final de una línea.

Esto es increíblemente útil para buscar palabras completas.

**Ejemplo: \bgato\b**

Este patrón busca la palabra "gato" de forma aislada.

| **Texto de prueba** | **¿Coincide con \bgato\b?** | **Explicación** |
|---|---|---|
| El **gato** negro. | Sí | "gato" está rodeado de espacios. |
| Me gusta el **gato**. | Sí | "gato" está rodeado de un espacio y un punto. |
| Los **gato**s son ágiles. | No | La 's' está pegada a "gato", no hay límite. |
| Estoy a**gato**pado. | No | "gato" no está en un límite de palabra. |

| | | |
|---|---|---|
| **gato** | Sí | La palabra está en su propia línea (inicio y fin). |

Sin \b, el patrón gato coincidiría en todos los casos anteriores, lo que a menudo no es el comportamiento deseado.

### Clases de Caracteres y Rangos

Las clases de caracteres nos permiten definir un conjunto de caracteres de los cuales **solo uno** necesita coincidir. Se definen usando corchetes [].

**Conjuntos: []**

Cualquier carácter que pongas dentro de los corchetes será parte del conjunto. El motor de REGEX intentará hacer coincidir **uno** de los caracteres del conjunto.

Ejemplo: **gr[ai]s**

Este patrón buscará la letra 'g', seguida de 'r', seguida de 'a' **o** 'i', y finalmente 's'.

- Coincide con "**gras**a".
- Coincide con "**gris**".
- No coincide con "grosor".

Ejemplo: **v[aeiou]l**

Este patrón coincidirá con "val", "vel", "vil", "vol" o "vul".

Conjuntos Negados: [^]

Si el primer carácter dentro de los corchetes es un acento circunflejo ^, el conjunto se niega. Esto significa que coincidirá con cualquier carácter que **no** esté en el conjunto.

Ejemplo: **q[^u]**

Este patrón buscará una 'q' que no esté seguida por una 'u'.

- Coincide con "**q a**" (el espacio no es 'u').
- Coincide con "**Iraq**" (el final de la línea no es 'u').
- No coincide con "**que**".

**Importante:** Dentro de [], el ^ solo tiene este significado especial si es el primer carácter. Si no, es un carácter literal. Por ejemplo, [a^b] coincide con 'a', '^' o 'b'.

Rangos: -

Para evitar escribir listas largas de caracteres secuenciales (como todos los números o todas las letras), podemos usar un guion - para definir un rango.

- [0-9]: Coincide con cualquier dígito del 0 al 9.
- [a-z]: Coincide con cualquier letra minúscula de la 'a' a la 'z'.
- [A-Z]: Coincide con cualquier letra mayúscula de la 'A' a la 'Z'.

Podemos combinar rangos y caracteres individuales en el mismo conjunto.

Ejemplo: [A-Za-z0-9_]
Este patrón tan común coincide con cualquier letra mayúscula, cualquier letra minúscula, cualquier dígito o un guión bajo. Es decir, un "carácter de palabra" típico.

Clases de Caracteres Predefinidas

Debido a que ciertos conjuntos de caracteres (como los dígitos o los espacios) se usan con mucha frecuencia, REGEX proporciona "atajos" o clases predefinidas para ellos. Estos atajos hacen que los patrones sean más cortos y legibles.

| Atajo | Equivalente | Descripción |
|---|---|---|
| \d | [0-9] | Cualquier dígito. |
| \D | [^0-9] | Cualquier carácter que no sea un dígito. |
| \w | [A-Za-z0-9_] | Cualquier carácter de palabra (alfanumérico + _). |
| \W | [^A-Za-z0-9_] | Cualquier carácter que no sea de palabra. |
| \s | [ \t\r\n\f\v] | Cualquier carácter de espacio en blanco (espacio, tabulador, nueva línea, etc.). |
| \S | [^ \t\r\n\f\v] | Cualquier carácter que no sea un espacio en blanco. |

**El comodín universal: . (Punto)**

El punto es uno de los metacaracteres más poderosos y simples. Coincide con **cualquier carácter individual**, excepto (normalmente) un salto de línea.

**Ejemplo: h.la**

Este patrón buscará una 'h', seguida de cualquier carácter, seguida de 'l' y 'a'.

- Coincide con "**hola**".
- Coincide con "**hila**".

- Coincide con "**h&la**".
- Coincide con "**h la**".

**Usando los atajos:**

En lugar de escribir [0-9][0-9][0-9], podemos escribir \d\d\d para buscar tres dígitos seguidos. Esto es mucho más limpio y fácil de leer.

Ejemplo: **\w\s\w**

Este patrón busca un carácter de palabra, seguido de un espacio en blanco, seguido de otro carácter de palabra.

- Coincide con "**a b**".
- Coincide con "**1 2**".
- Coincide con "**R_t**" si hay un tabulador en medio.

**Cuantificadores: ¿Cuántas veces?**

Los cuantificadores nos permiten especificar cuántas veces debe aparecer un carácter, grupo o clase de caracteres para que se produzca una coincidencia. Se colocan inmediatamente después del elemento que cuantifican.

| Cuantificador | Nombre | Descripción |
|---|---|---|
| * | Asterisco (Cero o más) | El elemento anterior puede aparecer 0 o más veces. |
| + | Más (Uno o más) | El elemento anterior debe aparecer 1 o más veces. |

| | | |
|---|---|---|
| ? | Interrogación (Cero o uno) | El elemento anterior es opcional (0 o 1 vez). |
| {n} | Llaves (Número exacto) | El elemento anterior debe aparecer exactamente n veces. |
| {n,} | Llaves (Mínimo) | El elemento anterior debe aparecer n o más veces. |
| {n,m} | Llaves (Rango) | El elemento anterior debe aparecer entre n y m veces. |

**Ejemplos:**

1. **Asterisco *: ho*la**
   - Coincide con "**hla**" (cero 'o').
   - Coincide con "**hola**" (una 'o').
   - Coincide con "**hooooooola**" (muchas 'o').
2. **Más +: ho+la**
   - No coincide con "hla".
   - Coincide con "**hola**".
   - Coincide con "**hooooooola**".
3. **Interrogación ?: colou?r**
   - Este patrón es útil para palabras con ortografía variable (británica/americana).
   - Coincide con "**color**" (cero 'u').
   - Coincide con "**colour**" (una 'u').
4. **Llaves (Número exacto) {n}: \d{4}**
   - Busca una secuencia de exactamente 4 dígitos.
   - Coincide con "**1984**".
   - No coincide con "123" ni "12345".

5. **Llaves (Rango) {n,m}: \w{3,5}**
    - Busca una palabra con una longitud de entre 3 y 5 caracteres.
    - Coincide con "**sol**", "**luna**", "**casas**".
    - No coincide con "la" ni "planeta".

Combinar clases de caracteres y cuantificadores es la base del poder de REGEX. Por ejemplo, \d+ significa "uno o más dígitos", un patrón muy común para extraer números de cualquier longitud de un texto.

**Agrupación, Captura y Alternancia**

Hasta ahora, hemos aplicado cuantificadores a caracteres individuales. ¿Pero qué pasa si queremos aplicarlos a una secuencia de caracteres? Para eso usamos los grupos.

Agrupación y Captura: ()

Los paréntesis tienen dos funciones principales en REGEX:

1. **Agrupar:** Permiten tratar una secuencia de caracteres como una sola unidad.
2. **Capturar:** Almacenan el texto que coincide dentro del grupo para su uso posterior (por ejemplo, en una sustitución).

Ejemplo de Agrupación: **(ha)+**

Aquí, el cuantificador + se aplica a todo el grupo (ha).

- Coincide con "**ha**".
- Coincide con "**haha**".
- Coincide con "**hahahaha**".
- No coincide con "h" ni "ahah".

Ejemplo de Captura: **(\d{2})-(\d{2})-(\d{4})**

Este patrón busca una fecha en formato DD-MM-YYYY.

- Coincide con "**25-12-2023**".

Lo más importante es que crea tres **grupos de captura**:

- **Grupo 1:** Contendrá "25".
- **Grupo 2:** Contendrá "12".
- **Grupo 3:** Contendrá "2023".

Esta capacidad de extraer partes específicas de una coincidencia es fundamental en la programación.

Alternancia (OR): |

La barra vertical | actúa como un operador "OR". Permite especificar varias alternativas posibles en un punto del patrón.

Ejemplo: **gato|perro**

Este patrón buscará la palabra "gato" **o** la palabra "perro".

- Coincide con "Me gusta mi **gato**".
- Coincide con "El **perro** ladra".

La alternancia se usa a menudo dentro de un grupo para limitar su alcance.

Ejemplo: **Me gusta el (café|té) con leche**

Este patrón es más específico y eficiente que Me gusta el café con leche|Me gusta el té con leche.

- Coincide con "**Me gusta el café con leche**".
- Coincide con "**Me gusta el té con leche**".

**Cómo Usar REGEX en entornos reales**

La sintaxis de los patrones REGEX es bastante estándar, pero la forma de implementarlos varía ligeramente según la herramienta o el lenguaje de programación.

1. En Editores de Texto (Visual Studio Code)

La mayoría de los editores de código modernos tienen soporte para REGEX en sus funciones de búsqueda.

**Cómo usarlo en VS Code:**

1. Presiona Ctrl+F (o Cmd+F en Mac) para abrir la barra de búsqueda.
2. Introduce tu patrón REGEX en el campo de búsqueda.
3. Haz clic en el icono .* (Usar Expresión Regular) para activar el modo REGEX.

Ejemplo Práctico:
Se quieren encontrar todas las líneas de un archivo CSS que definen un color, ya sea en formato hexadecimal (#RRGGBB) o rgb().

- **Patrón:** color:.*(#|rgb)
- **Explicación:**
  - color:: Busca el texto literal "color:".
  - .*: Coincide con cualquier carácter (el . ), cero o más veces (el *).
  - (#|rgb): Un grupo que busca un # o el texto rgb.

Esto resaltará instantáneamente todas las líneas relevantes en el archivo.

2. En Lenguajes de Programación (JavaScript)

En JavaScript, las expresiones regulares son un tipo de objeto de primera clase. Se pueden crear de dos formas:

- **Literal:** /patron/flags (la forma más común).
- **Constructor:** new RegExp("patron", "flags").

Las flags (banderas) son modificadores opcionales (ej: i para ignorar mayúsculas/minúsculas, g para búsqueda global).

**Métodos comunes:**

- **test() (del REGEX):** Devuelve true si hay una coincidencia, false si no. Ideal para validaciones.
  const texto = "Hola mundo";
  const patron = /Hola/;

```
if (patron.test(texto)) {
    console.log("Se encontró una coincidencia."); // Esto se ejecutará
}
```

- **match() (del String):** Devuelve un array con las coincidencias o null.
  ```
  const texto = "Mi número de teléfono es 123-456-7890.";
  const patron = /\d{3}-\d{3}-\d{4}/;
  const resultado = texto.match(patron);

  console.log(resultado[0]); // Muestra "123-456-7890"
  ```

- **replace() (del String):** Busca un patrón y lo reemplaza.
  ```
  let texto = "El cielo es azul.";
  const patron = /azul/;
  const nuevoTexto = texto.replace(patron, "rojo");

  console.log(nuevoTexto); // Muestra "El cielo es rojo."
  ```

3. En Lenguajes de Programación (Python)

En Python, la funcionalidad de REGEX se gestiona a través del módulo estándar **re**. Primero, hay que importarlo:

```
import re
```

Uso de Cadenas "Raw" (crudas):
Es una buena práctica definir los patrones REGEX usando el prefijo r antes de las comillas (ej: r"\d+"). Esto le dice a Python que no interprete las barras invertidas \ como secuencias de escape de Python, evitando conflictos con la sintaxis de REGEX.

**Funciones comunes:**

- **re.search(patron, texto):** Busca la **primera** aparición del patrón en el texto. Devuelve un objeto de coincidencia si la encuentra, o None si no.

```
import re

texto = "El precio del producto es $19.99"
patron = r"\d+\.\d{2}" # \d+ -> uno o más dígitos, \. -> un punto
literal, \d{2} -> dos dígitos

coincidencia = re.search(patron, texto)

if coincidencia:
    print(f"Precio encontrado: {coincidencia.group(0)}") # Muestra
"Precio encontrado: 19.99"
```

- **re.findall(patron, texto):** Busca **todas** las apariciones no superpuestas del patrón y las devuelve como una lista de cadenas.
  import re

  texto = "Los códigos son A123, B456 y C789."
  patron = r"[A-Z]\d{3}" # Una letra mayúscula seguida de 3 dígitos

  codigos = re.findall(patron, texto)
  print(codigos) # Muestra ['A123', 'B456', 'C789']

- **re.sub(patron, reemplazo, texto):** Sustituye las coincidencias del patrón por el texto de reemplazo.
  import re

  texto = "Por favor, contactar a juan.perez@email.com para más info."
  patron = r"[\w.-]+@[\w.-]+\.\w+"
  texto_anonimizado = re.sub(patron, "[CORREO OCULTO]", texto)

  print(texto_anonimizado)
  # Muestra "Por favor, contactar a [CORREO OCULTO] para más info."

4. En Herramientas online

Para aprender, depurar y probar expresiones regulares, las herramientas online son un recurso invaluable. Permiten ver en tiempo real qué partes del texto coinciden con tu patrón y suelen ofrecer explicaciones detalladas.

**regex101.com**

Regex101 es una herramienta muy usada que ofrece:

- **Un editor de REGEX** con resaltado de sintaxis.
- **Un campo para el texto de prueba** donde puedes ver las coincidencias en tiempo real.
- **Una explicación detallada** de cada parte de tu patrón en lenguaje natural.
- **Una referencia rápida** de la sintaxis de REGEX.
- Soporte para diferentes "sabores" de REGEX (PCRE, Python, JavaScript, etc.).

**Cómo usarla:**

1. **Pega el patrón** en el campo "REGULAR EXPRESSION".
2. **Pega el texto** en el campo "TEST STRING".
3. **Analiza los resultados:** Las coincidencias se resaltarán. A la derecha, se verá un panel que desglosa tu patrón y te explica qué hace cada componente. Es una forma excelente de aprender y encontrar errores.

Otra alternativa popular es **regexr.com**, que ofrece una experiencia similar. Usar estas herramientas acelera drásticamente la curva de aprendizaje.